Tesselonix Assessment

Application Security Audit & Network Troubleshooting

Target: Microblog a975ef64864354867c88e0ed3a17ba7d17dca752

Prepared 30 October 2026. AI assistance disclosed in TOOLS_AND_AI.md. Raw evidence and executable source are in the accompanying repository.

Repository: https://github.com/devashishsaxen/tesselonix-security-network-assessment

Tools and AI disclosure

Codex (OpenAI coding assistant) read the assignment and supplied architecture, reviewed the pinned source, wrote the lab configuration, fixes, tests, evidence runner and reports, and executed the verification. The candidate must review, understand and personally re-run this work before representing it as their own understood assessment. No claim is made that the candidate already did so.

Tools used: Git and GitHub CLI; Docker Desktop / WSL2 / Compose; Python 3.11; PyMuPDF to read the supplied PDFs; pytest; Flask-Limiter; nmap; iproute2 / ss; iptables; netcat; tcpdump; PostgreSQL / SQLAlchemy; Playwright browser automation; ReportLab for the printable report. Runtime tools execute inside local containers.

Verification performed by the assistant is recorded under `evidence/`: live HTTP attempts, controlled JWT forgery checked by the app's real verifier, upstream and security tests, host and rogue connection attempts, required app DB queries, firewall counters, packet inspection and tests after recreation. Screenshots are separate UI evidence. Secrets are replaced with `[REDACTED]` or never emitted.

Suggestions from the architecture that were changed or rejected:

- Do not claim that a dual-homed application is a router, or that only one node is dual-homed: the admin node is too. IP forwarding is explicitly disabled. - Do not assert that a router failed on this machine; no such diagnosis was made. - Do not describe a default route as 'no route at all'. Record actual route tables and distinguish lack of direct subnet attachment from filtering. - Do not publish the vulnerable database on all host interfaces; use loopback only. - Do not include a plaintext throwaway database password in evidence: the assignment excludes credentials, so even lab passwords are random and ignored. - Do not rely on scanner labels or a UDP `nc` exit status: assert responses, inspect traffic, and use counters for the directly connected rogue source. - Use one repository with a baseline tag instead of a nested `.git` directory; a clean normal clone must include the target code and reproduce both builds. - Keep the reflection honest about assistant execution and candidate review.

Primary references consulted:

- Microblog source: https://github.com/miguelgrinberg/microblog/tree/a975ef64864354867c88e0ed3a17ba7d17dca752 - Flask-Limiter initialization, decorators and storage limitations: https://flask-limiter.readthedocs.io/en/stable/ - Docker Compose networking, service names and published/container ports: https://docs.docker.com/compose/how-tos/networking/

The two supplied PDFs are requirements/design inputs, not evidence that a finding exists. Findings are established by source review and executable local proofs.

Application security findings

Scope: Microblog at `a975ef64864354867c88e0ed3a17ba7d17dca752`, cloned from https://github.com/miguelgrinberg/microblog. Test only the isolated local instance. No public service testing. Severity is qualitative and depends on deployment.

A-1 — Authentication attempts are not throttled (Medium)

Root cause: `app/api/tokens.py` places Basic authentication on POST `/api/tokens` without a rate limit. Web login and reset routes similarly have no throttling. An attacker can perform repeated online guesses; this proof establishes absence of throttling over the observed burst, not that an unknown password was cracked.

Reproduce: `python scripts/verify.py app`. The runner creates a disposable account, proves valid authentication first, sends 20 wrong passwords from one peer, then tries the real password. Baseline returns twenty 401s and then 200. Fixed returns nine 401s and eleven 429s after the first successful request consumes one slot; the final correct attempt is also 429. See `evidence/app/a1_baseline.txt` and `a1_fixed.txt`. Tokens and credentials are intentionally omitted.

Fix: initialize Flask-Limiter and decorate **outside** `login_required`, ensuring failed authentication is counted too. Token issuance and web login POSTs have 10/minute per-peer limits; reset POST routes have 5/minute. GET login remains accessible. Do not trust arbitrary `X-Forwarded-For`; the regression test varies this header and confirms it cannot bypass the local peer bucket.

Tests: `test_token_throttling_and_no_forwarded_ip_bypass` and parametrized web POST tests in `test_security.py`. A separate peer still authenticates. Upstream model tests establish that unrelated core behavior remains intact.

Remaining limitations: in-memory limits are a lab mitigation with one worker, reset on restart, and can be bypassed by distributed peers. Per-peer limits may affect users sharing NAT. For production use shared persistent storage, account and peer policies, MFA, logging and carefully configured trusted proxies. This change does not claim complete protection from credential stuffing.

A-2 — Public fallback signing key enables forged reset tokens (High)

Root cause: `config.py` silently uses a known source-code string if the environment key is absent/empty. `User.verify_reset_password_token` trusts HS256 JWTs signed with that key. Under that configuration, knowledge of a user ID permits generation of an accepted password-reset token, with potential account takeover.

Reproduce: the same runner starts baseline **without a supplied key**, signs a short-lived reset JWT for the disposable audit user using the public fallback, and invokes the real `User.verify_reset_password_token` in the running app. The forged token resolves to the audit account before hardening and is rejected afterward; a genuine reset token still succeeds. No other user's data is accessed and the proof does not actually change a password. See `evidence/app/a2_baseline.txt` and `a2_fixed.txt`. This is conditional on default-key deployment, not a claim that all installations are exploitable.

Fix: read the key from environment only. Production app creation rejects absent, short and known-default keys before extension initialization; importing modules does not throw. Test/debug mode generates a fresh ephemeral key where needed. `scripts/setup.py` generates a private random 64-character key. Length checks cannot establish entropy; use the provided generator or a secret manager.

Related defense in depth: set session and remember cookies to Secure, HttpOnly, SameSite=Lax. Baseline had HttpOnly already; Secure and SameSite were missing. Cookie flags do not repair a known signing key or provide HTTPS themselves. Use TLS on real deployments; localhost browser behavior is a demo convenience.

Tests cover refused production starts, ephemeral test keys, rejected forgery, valid genuine reset tokens, and flags on **both** session and remember cookies after a successful login. Evidence includes actual cookie attributes with values redacted, plus `evidence/app/fail_closed.txt` for a fresh container without a key.

Operational follow-up: replace any previously exposed signing key, invalidate old sessions/reset tokens and assess access logs. A randomly generated key persists in ignored `.env` for this lab; use managed secret injection in production.

Scope decisions

Open registration is an intended feature and was not reported as a vulnerability. SQLAlchemy ORM use alone neither proves nor disproves injection safety; no SQL injection claim was established. Old dependency pins remain a review item; no scanner output is being presented as a verified exploit. Optional upstream services and production TLS/proxy deployment are outside the exercised scope.

Network assessment

See network-diagram.svg. The runtime inventory and route tables are in `evidence/network/addressing_routes.txt` and `port_inventory.txt`.

```
| Node | Edge 172.28.10.0/24 | Internal 172.28.20.0/24 | Role |
|---|---|---|---|
| Docker gateway | .1 | .1 (configured bridge) | Bridge boundaries |
| app | .10 | .11 | HTTP/API and DB client |
| db | — | .10 | PostgreSQL / UDP echo / host firewall |
| netops | .60 | .60 | Authorized administration/tests |
| client | .50 | — | Untrusted edge peer |
| rogue | — | .99 | Unlisted directly connected peer |
```

Both subnets are /24. The internal bridge has `internal: true`, so it does not offer ordinary external forwarding. Default routes and gateway behavior are platform-generated; consult the captured `ip route` and app `/proc/net/route` rather than assuming each node uses both configured gateways.

App and netops are dual-homed but **not routers**; `ip_forward=0` is explicit. The app mediates HTTP-to-database operations in userspace. Docker bridges isolate the segments. The edge client has a default route but no directly connected internal route; inter-network filtering prevents access. We demonstrate two subnets and their boundary, not successful IP routing through a router container. The assignment's router preference is 'where feasible'; this limitation is disclosed.

Inventory and policy

```
| Service | Listener | Protocol | Required access |
|---|---|---|---|
| Gunicorn/Flask | app 0.0.0.0:5000 | TCP / HTTP | Client, local browser |
| PostgreSQL | db 0.0.0.0:5432 and IPv6 listener | TCP | App and authorized admin |
| Synthetic UDP echo | db 0.0.0.0:5353 | UDP | Authorized admin demonstration only |
| Docker embedded DNS | Container-local resolver | TCP/UDP 53 | Container name resolution |
```

IPv6 application-network addressing is not enabled. The host publishes only `127.0.0.1:5000`; no DB or UDP mapping in secure mode. A bind address inside a container is not the host's publication address. UDP port 5353 is a synthetic unicast echo service here, **not an implementation of mDNS**.

The port-only nmap scan labels TCP 5000 `upnp` using its port-name database; that is not a service fingerprint. The configured Gunicorn listener and real HTTP responses establish that this lab's service is Flask HTTP, not UPnP.

```
| Source → destination | app TCP 5000 | db TCP 5432 | db UDP 5353 |
|---|---|---|---|
| client edge | Allow | Deny | Deny |
| app internal | — | Allow | Deny |
| netops internal | Allow | Allow | Allow |
| rogue internal | No requirement | Deny | Deny |
| host loopback | Allow | Deny | Deny |
```

The network test automates client/app/admin/rogue and host checks. The DB default DROP policy also denies app UDP by configuration; that specific matrix cell is not separately probed. No claim that every port on every node was exhaustively scanned. Netops is a trusted dual-homed administration node, not an untrusted peer.

Firewall: allow loopback and established/related replies; allow TCP 5432 from `.11` and `.60`; allow UDP 5353 only from `.60`; default-drop INPUT. Docker may install resolver-related rules in the namespace as well. Capture effective rules with `iptables -S INPUT`, not only the script. No host firewall is modified.

N-1 — Unnecessary database access (Medium, controlled misconfiguration)

This weakness is **deliberately introduced in a separate local lab configuration**, not attributed to the upstream Microblog repository. The insecure override publishes DB TCP 5432 to host loopback port 15432 and makes the DB firewall permissive. It also disables `internal: true` in the controlled before-state so Docker permits host publication.

Secure mode restores internal isolation as well as the firewall. The host can connect and the rogue peer reaches the service. An existing host service already listens on 5432, so 15432 avoids interfering with it. This grants an unnecessary attack surface even though PostgreSQL authentication still applies. No unauthorized database read is claimed. The loopback restriction limits risk while preserving the unwanted-host-access demonstration.

Reproduce and fix: `python scripts/verify.py network` applies the override, tests access, tears it down without deleting volumes, then uses the secure file alone. It removes publication and reinstates source allowlists. Host connections fail, rogue TCP times out, UDP has no reply, while the app runs a real `SELECT 1` and the edge client gets HTTP 200. See `n1_before.txt`, `n1_host_after.txt`, `access_tests.txt` and `firewall_counters.txt`.

Rules reapply in the DB entrypoint on each creation. The runner force-recreates the stack and verifies host/rogue denial and successful API authentication again: `redeploy_verified.txt`. This proves persistence of declarative configuration, not merely a one-off command left in a container namespace.

TCP/UDP diagnosis and interpretation

The pcap and decoded transcript show TCP SYN, SYN/ACK and ACK on authorized traffic, plus attempted SYN retransmissions with no success for the rogue source. A real app SQL query separately establishes usable database communication; TCP reachability alone does not prove database credentials or a successful query.

UDP has no handshake. Netops sends a synthetic datagram and receives the echo; client/rogue probes receive no echo. A timeout alone cannot distinguish packet loss, filtering, wrong route or an unavailable service. The authorized control, directly connected rogue source, captured requests and increasing default-drop counters jointly support the firewall conclusion. `nc -u` success exit alone is not evidence of a delivered datagram.

`traffic.pcap` retains zero-payload TCP segments and synthetic UDP only, using a BPF length filter. It intentionally excludes TCP auth/query payloads. Capture at the DB sees incoming packets before INPUT dropping, so seeing a rogue packet in the trace does not mean it was delivered to PostgreSQL. Edge packets may be blocked before reaching the DB capture; absence alone does not locate the drop.

Limitations: static source IPs are not identities against a privileged Docker administrator; Docker/host compromise is outside this lab. Internal DB traffic is not a claim of encrypted production transport. A production design needs TLS, stronger identity controls, managed secrets and maintained dependencies.

Verification and reproducibility

Run `python scripts/setup.py`, `python scripts/build.py`, then `python scripts/verify.py` from a clone with the baseline tag. The runner uses only the local app and its dedicated Docker subnet addresses, with assertions on expected outcomes. Commands for individual container tests are embedded in the transcripts; the exact automation is version controlled in `scripts/verify.py`.

Application before/after

```
| Check | Baseline | Hardened | Evidence |
|---|---|---|---|
| Valid token authentication first | 200 | 200 | `app/a1_*.txt` |
| Twenty wrong passwords after valid request | 20 × 401 | 9 × 401, 11 × 429 | `app/a1_*.txt` |
| Correct password after burst | 200 | 429 | `app/a1_*.txt` |
| Reset JWT signed with public default | Accepted | Rejected | `app/a2_*.txt` |
| Genuine reset JWT | Accepted | Accepted | `app/a2_*.txt` |
| Session Secure / SameSite | False / None | True / Lax | `app/a2_*.txt` |
| Fresh production startup without key | Insecure fallback | RuntimeError | `app/fail_closed.txt` |
| Tests | Upstream suite passes | Upstream + security suite pass | `app/tests_*.txt` |
```

The one earlier valid request consumes a slot; this explains why the wrong-password burst has nine rather than ten pre-limit responses. Limits count all POST attempts, not only failures. They reject at the limiter before credential verification.

The forgery proof checks the real verifier in the running application process with its effective configuration and real database user. It does not rely solely on decoding a self-generated JWT or comparing a config string. Unit tests additionally exercise legitimate web login and session/remember cookie flags.

Network before/after

Controlled insecure setup: combine `lab/docker-compose.net.yml` with `lab/docker-compose.net.insecure.yml`. The latter is **not** an override to leave enabled: the secure test removes the stack and recreates it with the first file only.

```
| Check | Insecure local lab | Hardened lab | Evidence |
|---|---|---|---|
| Host → localhost:15432 | TCP connects | Connection blocked | `network/n1_before.txt`,
`n1_host_after.txt` |
| Rogue → DB TCP 5432 | Connects | Timeout | `network/n1_before.txt`, `access_tests.txt` |
| Admin → DB TCP 5432 | — | Connects | `network/access_tests.txt` |
| App → DB query | — | SELECT 1 returns 1 | `network/access_tests.txt` |
| Admin → UDP 5353 | — | Echo received | `network/access_tests.txt` |
| Client/rogue → UDP 5353 | — | No reply | `network/access_tests.txt` |
| Edge client → HTTP login | — | HTTP 200 | `network/access_tests.txt` |
| Secure stack force-recreated | — | Host/rogue denied, API works | `network/redeploy_verified.txt` |
```

`network/traffic.pcap` and `traffic_decoded.txt` capture authorized TCP connection establishment and synthetic UDP request/reply, plus denied probes where visible at the DB. `firewall_counters.txt` supplies effective rules and counters. Empty-payload TCP filtering excludes credentials and SQL data; SYN/SYN-ACK/ACK remain visible.

The host originally had a service already using 5432; the lab therefore uses host **15432**. It never stops or changes the existing host service. The internal PostgreSQL service remains on **5432**. Source code and documentation use that distinction consistently.

Persistence, rollback and limits

App configuration is in source, dependency pins and image; DB access policy is in the entrypoint and Compose membership/publication. Recreating containers reasserts the controls. `.env` is generated once and retained privately, so restarts do not invalidate sessions by silently changing the key. The in-memory limiter bucket itself is not persistent and is explicitly a lab limitation.

Rollback for comparison uses the immutable baseline tag/image, or the explicitly insecure local override. Do not use either for production. `down` removes lab containers/networks while preserving volumes; `down -v` also erases lab data.

This establishes the tested outcomes on this Docker Desktop environment. It does not establish comprehensive app security, production TLS, exhaustive firewall coverage, resistance to a Docker administrator or candidate understanding. The candidate must rehearse the walkthrough before submission.

Browser verification (`evidence/app/browser_smoke.txt`) additionally exercised a real CSRF-protected login, remember cookie flags, post creation, persisted-post display on Explore, and redirecting an unauthenticated session. Three actual screenshots are under `evidence/screenshots/`. The browser smoke runner also verifies token issuance, bearer-authenticated API user listing (200), and rejection of the same API read without a token (401).

Clean-clone reproduction is recorded in `evidence/clean_clone.txt` and the actual image-build output in `evidence/clean_build.txt`. The full verifier was repeated from commit `9311b29` with fresh random credentials and separate project volumes; the final hardened app source is unchanged from that commit. Later changes add documentation, optional browser/API assertions, evidence serialization and package checks. `evidence/source-provenance.txt` records the byte-for-byte upstream match.

Technical reflection and candidate review

This is an assistant-generated engineering reflection on work executed in this session. It is not a statement of the candidate's personal experience or prior knowledge. Before submission, the candidate should re-run the demonstrations and replace or supplement this section with their own observations.

What required care

Maintaining a true vulnerable baseline matters: building both tags from whichever branch happens to be checked out would invalidate the comparison. The build script instead exports a preserved Git tag to a temporary context. A normal clone includes the vendored target, its attribution and both source states.

The effective signing key is more significant than a string in source. The proof uses the app's actual verifier and a real disposable user; the fixed app rejects the forged token and still accepts legitimate tokens. The secret check belongs in the factory so imports and test configuration remain usable.

Network silence is ambiguous. A directly connected unauthorized node, an authorized control, packet capture and firewall counters together establish more than an isolated timeout. The dual-homed app is not a router; explicit terminology avoids claiming that IP forwarding was demonstrated when it was not.

An existing host service occupied PostgreSQL's usual port. Moving only the lab's temporary published port to 15432 preserved the internal service and avoided interfering with unrelated software. This is a real setup observation, distinct from the deliberately introduced DB exposure used for the assessment.

Lessons reflected in the implementation

Evidence should come from assertions against running software and be redacted at capture time. Cookie flags are useful but cannot compensate for a public signing key. A rate limit can protect one peer bucket while still having distributed and restart limitations. Static source allowlists are useful in a controlled container lab but should not be described as strong workload identity.

What to investigate next

Move rate-limit storage to shared Redis with failure handling and account-aware policies; add real TLS termination; rotate and manage signing keys outside files; review and update the old upstream dependency set; examine authorization and API token lifetime more deeply; add maintained CI supply-chain checks. A separate router/VM lab could demonstrate actual routed forwarding and traceroute without conflating application-layer mediation with a network router.

Candidate rehearsal

Personally run the full verifier, inspect one raw transcript and decode the pcap. Explain what an HTTP 401, HTTP 429, TCP SYN/ACK, UDP timeout and INPUT DROP counter each establish. Locate the corresponding fix, modify one allowed source in a lab copy, predict the effect and verify it. State clearly which work was AI-assisted and which observations you personally reproduced.

Live walkthrough and project explanation

Use this as a rehearsal guide after reviewing the implementation. Do not claim personal verification until you have actually run and understood it.

1. What runs where

The browser sends HTTP to host localhost:5000. Docker forwards this to Gunicorn, which serves the Flask Microblog app. Flask uses SQLAlchemy to query PostgreSQL over TCP 5432. The host does not need PostgreSQL or Python app packages installed. Web login uses Flask-Login sessions; API token creation uses HTTP Basic, and other API requests use a bearer token. Passwords are hashed by upstream code.

`app/microblog/config.py` supplies settings. `app/__init__.py` creates the app and extensions. `app/auth/routes.py` implements web auth; `app/api/tokens.py` implements token creation/revocation. `app/models.py` holds users/posts and reset-token logic. Alembic migrations create the database schema before Gunicorn starts.

2. See the working application

```
docker compose --env-file .env -f lab/docker-compose.net.yml ps
Start-Process http://localhost:5000
```

Register your own local user, log in, create a post and visit Explore. If the verifier ran, its disposable login is in `.build/demo-login.txt`. Keep that file private; it is deliberately excluded from Git and the submission ZIP.

3. Explain the application findings

A-1: Basic authentication checks credentials on every POST. Without a throttle, the endpoint accepts repeated attempts. The limiter now counts by the actual remote peer before authentication. A correct password is also blocked after that peer has exhausted its allowance. This is why a 429 is expected even for a valid password at the end of the burst. Memory storage is suitable only for this single-worker demonstration; multiple workers need shared storage.

A-2: a signing key is not a password hash. It lets the server recognize data it signed. If everyone knows that key, an attacker can sign a reset token too. The fix refuses unsafe production configuration and generates a private key locally. The forgery is rejected while a token signed by the app still works. Existing deployments using the public fallback must rotate their key and invalidate tokens.

Secure tells a browser to restrict cookies to secure transport (with browser localhost allowances); HttpOnly prevents ordinary JavaScript from reading them; SameSite=Lax restricts some cross-site cookie sending. None replaces server-side authorization, CSRF protection or a private signing key.

```
python scripts/verify.py app
docker run --rm tx-microblog:fixed pytest -q tests.py test_security.py
```

The app phase ends by stopping Task A. Start Task B again for the following demo.

4. Explain the network and prove the fix

```
python scripts/verify.py network
docker compose --env-file .env -f lab/docker-compose.net.yml exec db iptables -nvL INPUT
docker compose --env-file .env -f lab/docker-compose.net.yml exec client ip route
docker compose --env-file .env -f lab/docker-compose.net.yml exec netops nc -zv -w 3 172.28.20.10 5432
docker compose --env-file .env -f lab/docker-compose.net.yml exec rogue nc -zv -w 3 172.28.20.10 5432
```

The authorized connection succeeds; rogue times out. The verifier separately queries the DB through the app. PostgreSQL may listen on all container interfaces, but only allowed network sources can reach it. Removing a Docker host publication and applying a DB firewall are separate controls. The host test uses 15432 because an unrelated service already occupies host port 5432.

TCP performs SYN → SYN/ACK → ACK before application data. A handshake proves transport reachability, not a valid database login. UDP sends datagrams without a handshake; the admin echo reply establishes a round trip. No reply could mean several things, so use captures, the authorized control and counters together. The synthetic UDP 5353 echo is not DNS or mDNS.

```
docker compose --env-file .env -f lab/docker-compose.net.yml cp evidence/network/traffic.pcap
db:/tmp/review.pcap
docker compose --env-file .env -f lab/docker-compose.net.yml exec db tcpdump -nn -r /tmp/review.pcap
```

The two Docker bridges are separate subnets. The app and trusted admin attach to both, but IP forwarding is disabled. They do not act as routers. HTTP and DB requests are separate application connections; say so if asked about routing.

5. Explain evidence and trade-offs

Open the matching file under `evidence/` as you show each claim. Timestamped transcripts are actual outputs; `traffic.pcap` is machine-readable packet proof; screenshots show UI operation. The patch shows precisely what changed from the baseline. The reports explain meaning and limits rather than replacing raw proof.

Useful questions to rehearse: What happens after a worker restart? Why is IP-only throttling incomplete? Why does a UDP `nc` exit not prove delivery? Why use a rogue peer on the internal subnet? What does `internal: true` provide? Why can a packet appear in tcpdump and still be blocked? Which controls survive recreation?